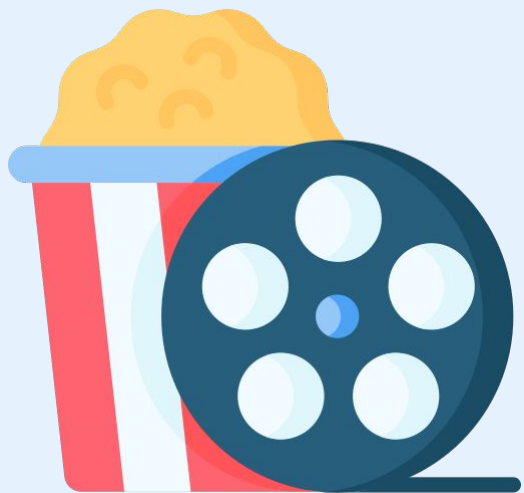
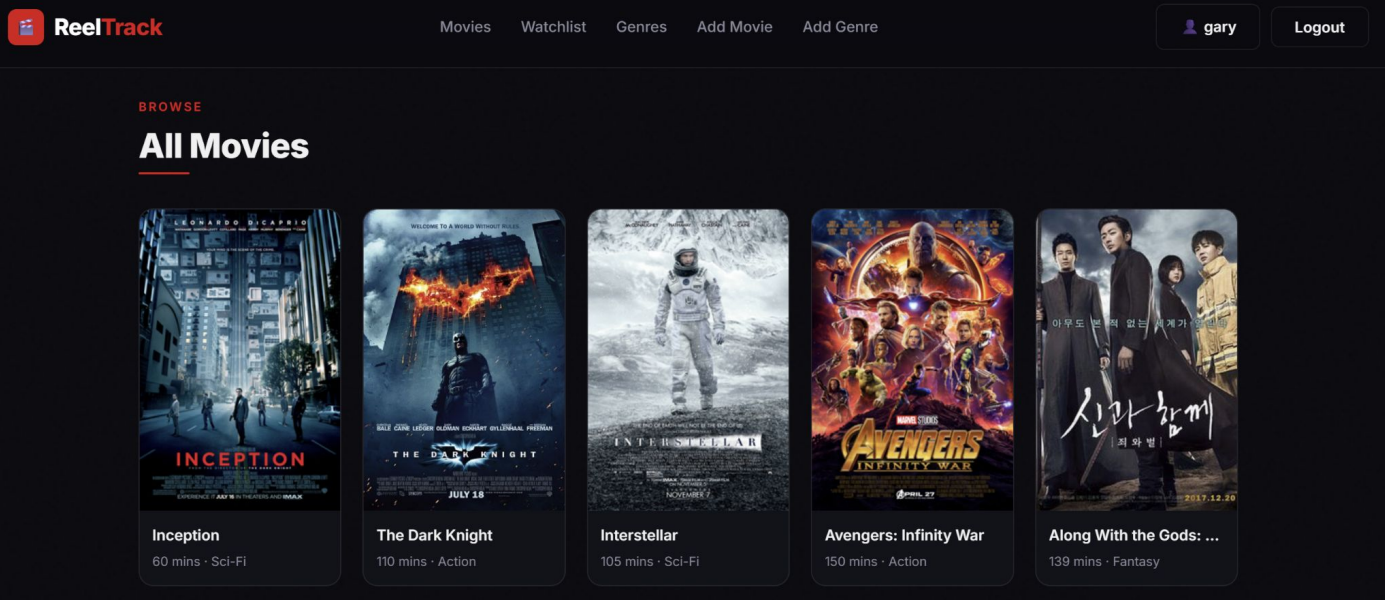




ReelTrack

Your Cinema. Your Story.

T6: Farhaan • Bing Xian • Jessie • Gary • Bryan • Sungwon



Our scenario

A movie application where users can manage their *watchlists*, *submit ratings and reviews*, and participate in *discussions* about an extensive catalog of movies, all while supported by comprehensive *user and genre* management



Features



Users

- Registration
- Authentication
- Update
- Delete



Movies

- Create (admin)
- Read (public)
- Update (admin)
- Delete (admin)



Genre

- Create (admin)
- Read (public)
- Update (admin)
- Delete (admin)



Reviews

- Create (logged-in)
- Read (public)
- Update (Owner only)
- Delete (Owner only)



Comments

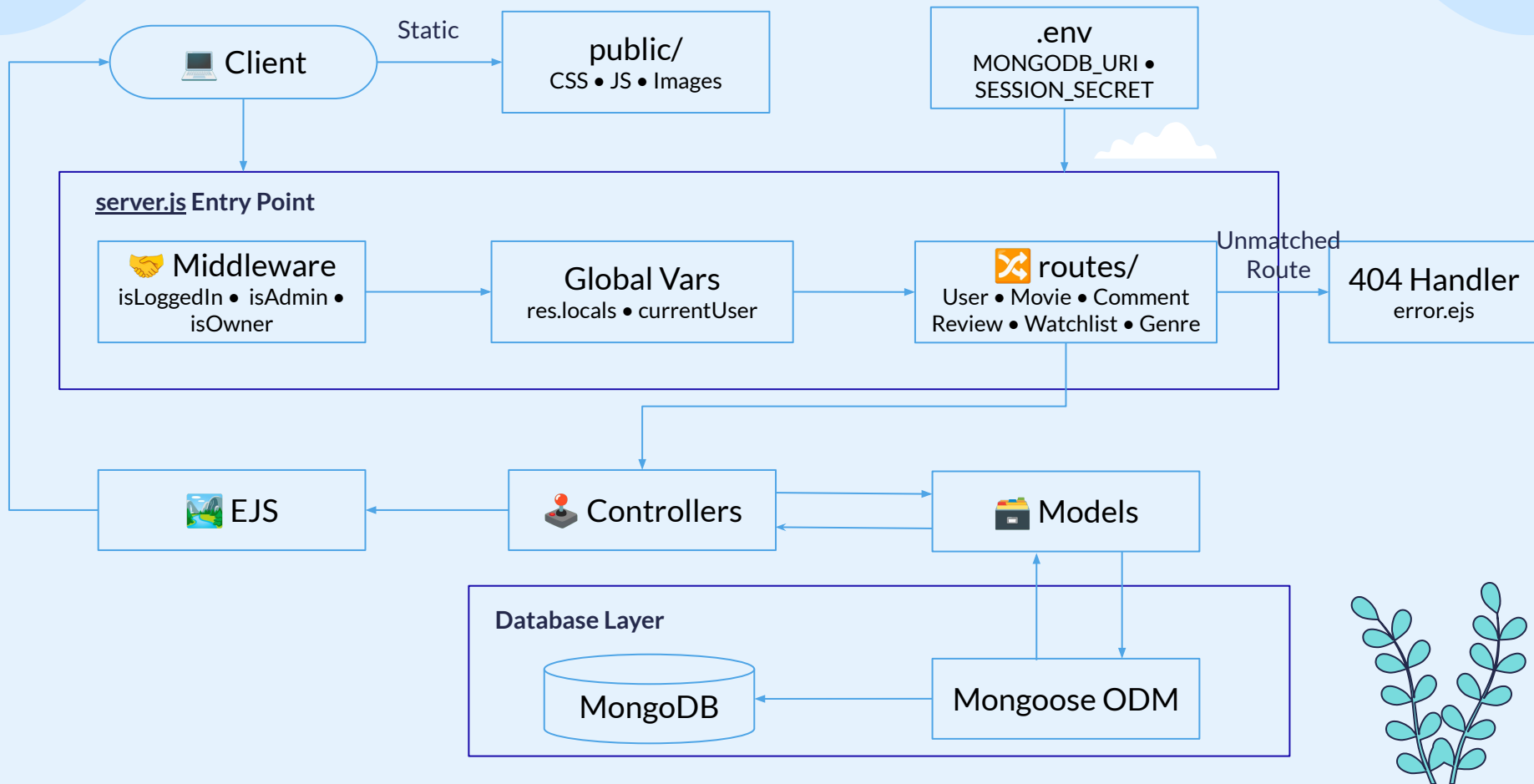
- Create (logged-in)
- Read (public)
- Update (Owner only)
- Delete (Owner only)

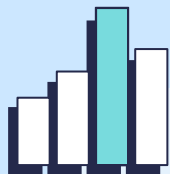


Watchlist

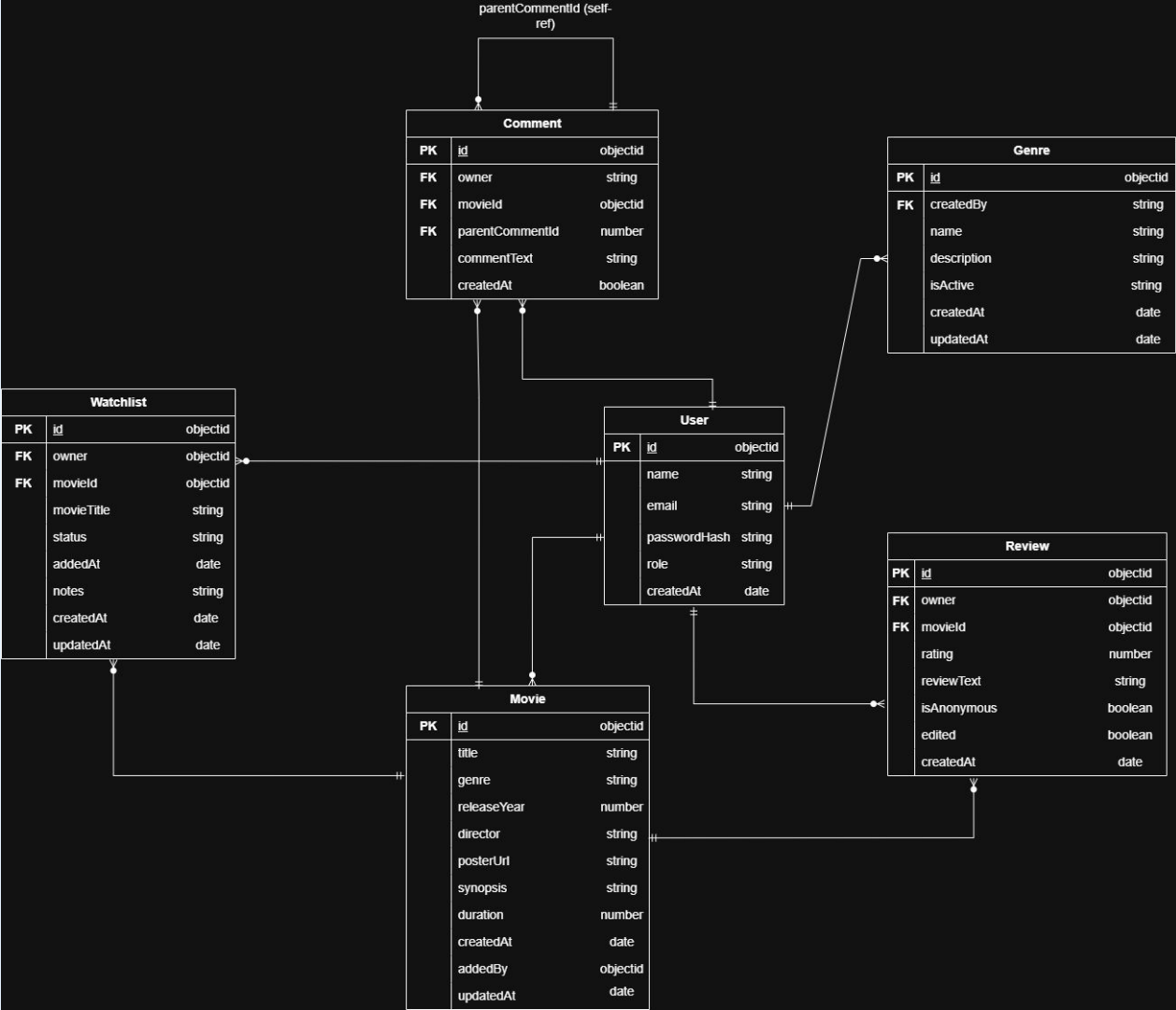
- Create (logged-in)
- Read (logged-in)
- Update (Owner only)
- Delete (Owner only)

System Architecture: MVC





Database Design





Live Demo

Movies & Reviews

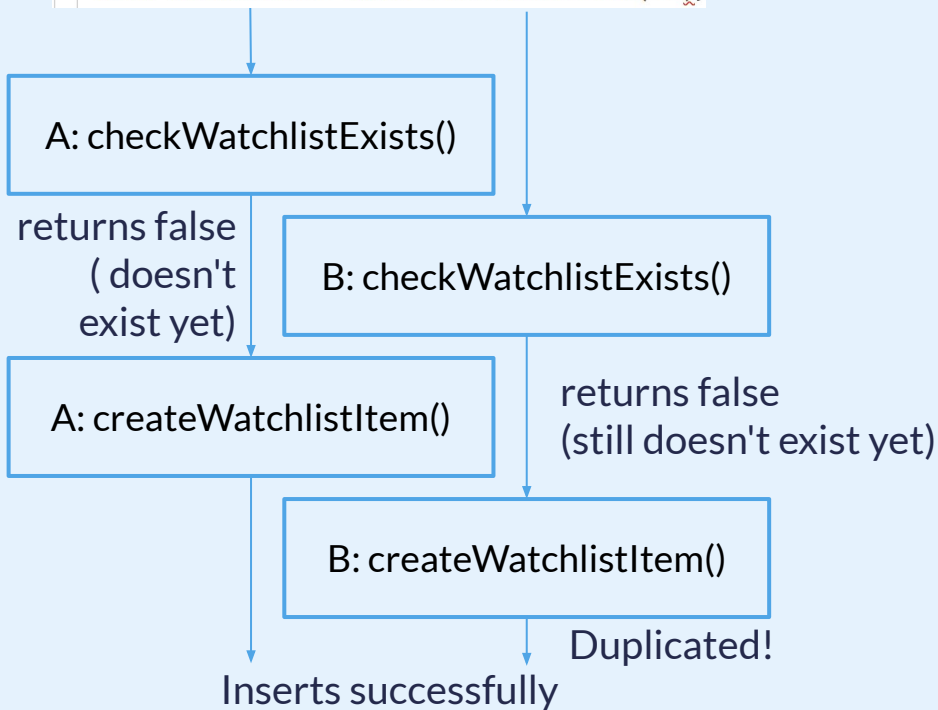


Learning Points

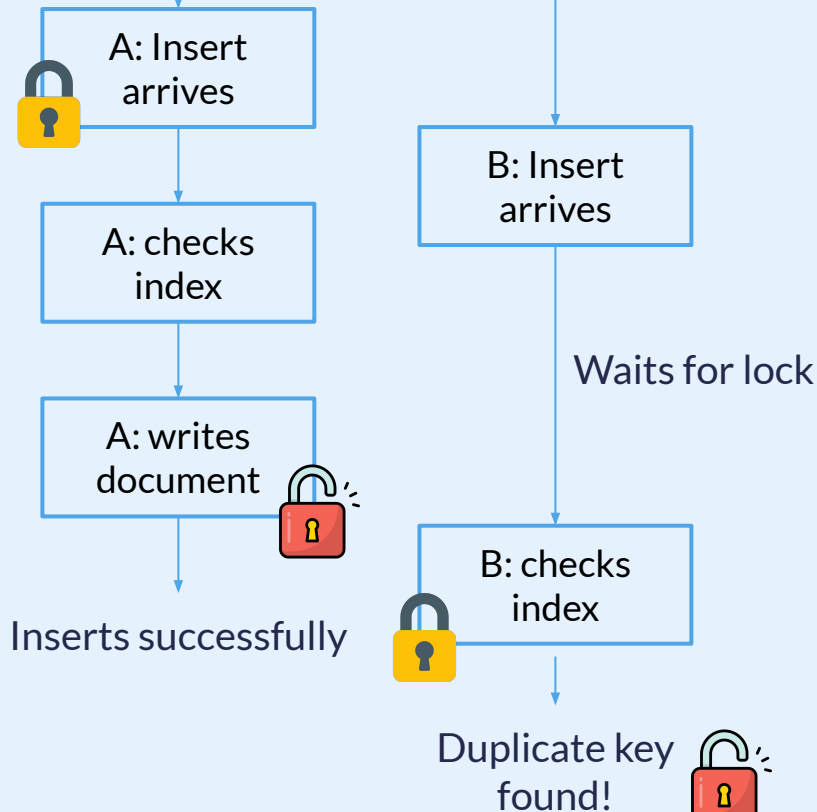
- Validations done in model layer needs to be enforced on update operations
- CRUD operations require consistent error handling to ensure application reliability
- MongoDB associations between models requires careful schema design for efficient data retrieval

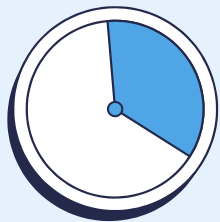
Role of Unique Index

```
async function create(req, res) {  
  try {  
    ...  
    const exists = await checkWatchlistExists  
      | (req.body.movieId, req.session.userId);  
    ...  
    const watchlistItem = await createWatchlistItem(...);  
  }  
}
```



watchlistSchema.index({ owner: 1, movieId: 1 },
{ unique: true });





Thanks!

Do you have any questions?

CREDITS: This presentation template was created by **Slidesgo**, including icons by **Flaticon**, infographics & images by **Freepik**

